FPT UNIVERSITY
SOFTWARE ENGINEER MAJOR
LAB211



FPT UNIVERSITY

HOMESTAY BOOKING MANAGEMENT SYSTEM REPORT

SUPERVISOR : Ms. Than Thi Ngoc Van
SUBJECT : LAB211
CLASS : SE2043

STUDENT : Nguyen Van Gia Binh (SE203555)

HO CHI MINH CITY 3/2026

Contents

Chapter 1 Introduction 2
1.1 Project Overview 2
1.2 Scope and Target Audience 2
Chapter 2 System Overview 3
Chapter 3 System Architecture 4
3.1 Layered Architecture 4
3.2 Class Diagram 4
Chapter 4 Software Pseudocode 6
4.1 Function 1: List all Developers 6
4.2 Function 2: Add new developer 7
4.3 Function 3: Search for a Developer by ID 8
4.4 Function 4: Update a Developer's salary by ID 9
4.5 Function 5: List all Developers by Language 10
4.6 Function 6: Add a new Project 11
4.7 Function 7: List all Projects by Developer (Grouped) 13
4.8 Function 8: Calculate Total Experience by Dev ID 14
4.9 Function 9: Remove a Developer by ID 15
4.10 Function 10: Sort Developers by Salary 16
4.11 Function 11: Save data to files 17
4.12 Function 12: Quit program 17
Chapter 6 Business Logic Architecture 18
6.1.1 Developer Management Logic 18
6.1.2 Project Management Logic 18
6.1.3 Reporting and Analytics 19
Chapter 7 Conclusion 20

Chapter 1 Introduction

1.1 Project Overview

The Software Developer Management Application is a console-based Java application. The application addresses the need for a structured system to manage software developer profiles, programming skills, and project assignments. In the rapidly evolving software development industry, IT companies and project managers need efficient tools to manage developer information, track project allocations, and evaluate work experience and salaries. This application provides a simplified yet functional solution for managing these core operations through a text-based menu interface.

1.2 Scope and Target Audience

The primary objective of this project is to build a fully functional Software Developer Management System that allows manager to:

- Manage Developers: Add, remove, update salaries, and search developers by ID or programming language.
- Manage Projects: Assign developers to new projects.
- Generate Reports: Calculate developer experience, list developer's assigned projects, and sort developers by salary.
- Persist Data: Save and load data to developers.txt and projects.txt for persistence.

Chapter 2 System Overview

The system operates as a menu-driven application with 12 main functions:

1. **List all Developers:** Display the information of all developers currently in the system.
2. **Add a new Developer:** Create a new software developer with validation (ID, Name, Programming Languages, Salary > 1000).
3. **Search for a Developer by ID:** Find and display the developer's information using their unique ID.
4. **Update a Developer's salary by ID:** Modify the salary of an existing developer by their ID.
5. **List all Developers by Language:** Find and display developers who know a specific programming language.
6. **Add a new Project:** Assign an existing developer to a new project with validation (future start dates, > 1 month duration).
7. **List all Projects by Developer (Grouped)**: Display all assigned projects grouping by every developer.
8. **Calculate Total Experience by Dev ID:** Calculate a specific developer's total project experience in months.
9. **Remove a Developer by ID:** Delete a developer from the system (only if they are not assigned to any projects).
10. **Sort Developers by Salary:** Display the list of developers sorted by their salary in ascending order.
11. **Save data to files**: Manually save all updated developer and project information to data files (developers.txt, projects.txt).
12. **Quit program:** Upon exit, the system shall prompt the user to see if they want to save changes.

Chapter 3 System Architecture

3.1 Layered Architecture

View: Main, Menu – Display the console menu and interact with the user.

Controller: Controller – Handle user choices, connect View to Service layer.

Service: DeveloperList, ProjectList, Iservice – Include business logic, validation, and data processing for each entity.

Model: Developer, Project – Define the data structure for objects.

File Utilities: FileUtils – Read/write data to files.

Utilities: Inputter, Validator – Provide input handling and validation tools for all layers.

3.2 Class Diagram

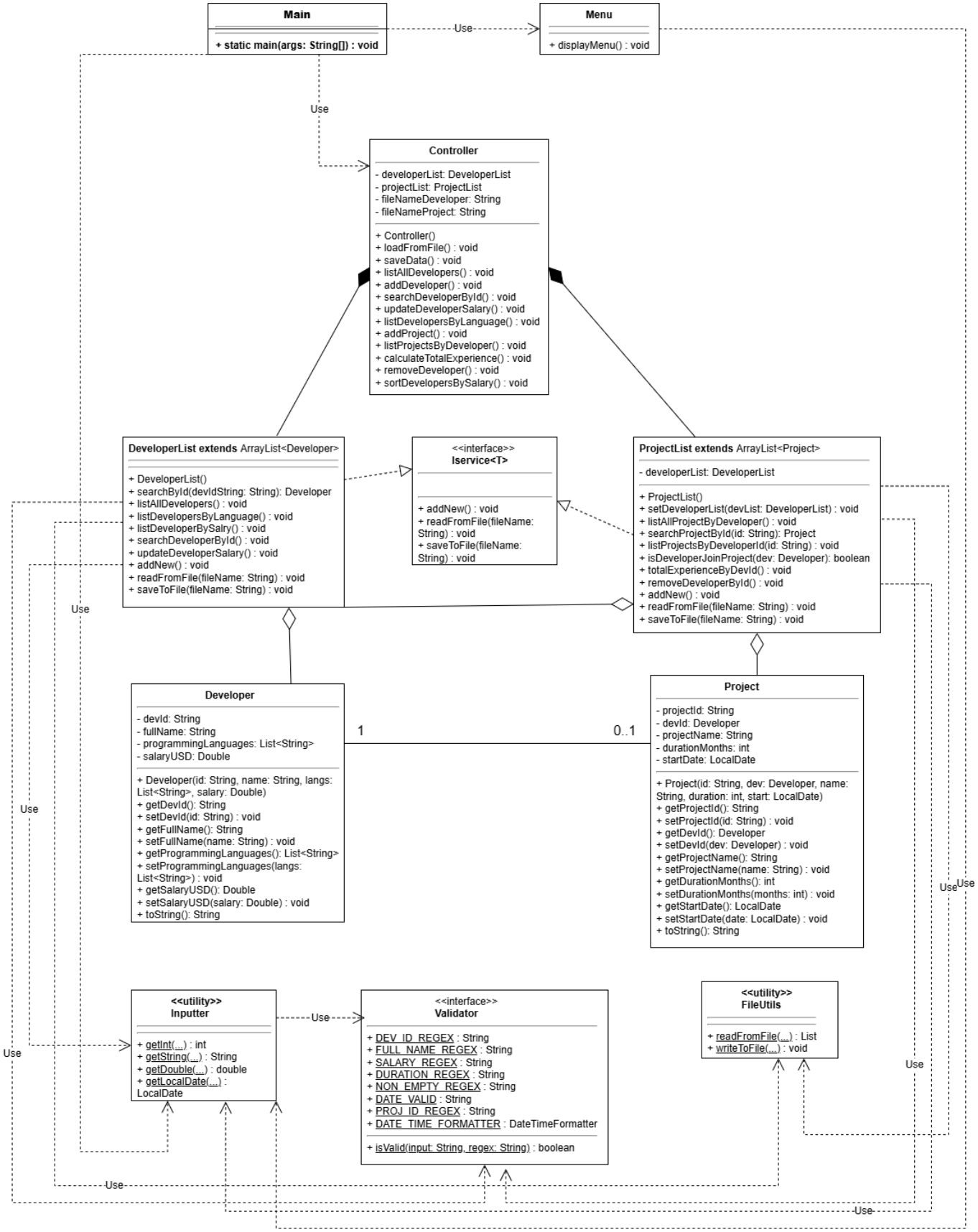


Figure 3.1: Detailed Class Diagram of the System

Chapter 4 Software Pseudocode

4.1 Function 1: List all Developers

```
1    ALGORITHM   List All Developers BEGIN
2
3         PRINT "--- List All Developers ---"
4
5         IF DeveloperList is empty THEN
6              PRINT "No developers found."
7              RETURN
8         END IF
9
10        PRINT "-------------------------------------------------------------------------------------------------"
11             PRINT   "|  ID                    |  Name                                          |  Languages
| Salary ($)     |"
12        PRINT "-------------------------------------------------------------------------------------------------"
13
14        FOR EACH dev IN DeveloperList DO
15             Join dev.programmingLanguages with ","
16             PRINT formatted row: dev.ID, dev.Name, dev.Languages, dev.Salary
17        END FOR
18
19        PRINT "-------------------------------------------------------------------------------------------------"
20
21   END
```

Listing 4.2: Pseudocode for list all developers

4.2 Function 2: Add new developer

```
1   ALGORITHM Add New Developer BEGIN
2
3       PRINT "--- Add New Developer ---"
4
5       REPEAT
6           INPUT devID (format: DEVXXX)
7           Search developer record by devID in developerList
8
9           IF devID format is invalid THEN
10              PRINT "Error: Invalid ID format! Format must be DEVXXX."
11          ELSE IF developer is found THEN
12              PRINT "Error: Developer ID already exists!"
13          ELSE
14              BREAK LOOP
15          END IF
16      END REPEAT
17
18      REPEAT
19          INPUT fullName (format: alphabet characters and spaces)
20
21          IF fullName format is invalid THEN
22              PRINT "Error: Invalid name format!"
23          ELSE
24              BREAK LOOP
25          END IF
26      END REPEAT
27
28      INPUT programmingLanguagesString (Enter programming languages, separated by comma)
29      Trim and split programmingLanguagesString into programmingLanguages list
30
31      REPEAT
32          INPUT salaryUSD (Must be numbers)
33
34          IF salaryUSD format is invalid OR salaryUSD < 1000 THEN
35              PRINT "Error: Salary must be a valid number and at least 1000!"
36          ELSE
37              BREAK LOOP
38          END IF
39      END REPEAT
40
41      Create new Developer object with (devID, fullName, programmingLanguages, salaryUSD)
42      Add the new Developer into developerList
43
44      PRINT "[Success] New Developer has been added successfully!"
45
46  END
```

Listing 4.2: Pseudocode for Add new developer

4.3 Function 3: Search for a Developer by ID

```
1   ALGORITHM Search Developer By ID BEGIN
2
3       PRINT "--- Search Developer By ID ---"
4
5       REPEAT
6           INPUT devID (format: DEVXXX)
7
8           IF devID format is invalid THEN
9               PRINT "Error: Invalid ID format! Format must be DEVXXX."
10          ELSE
11              BREAK LOOP
12          END IFs
13      END REPEAT
14
15      Search developer record by devID in developerList
16
17      IF developer is NOT found THEN
18          PRINT "Error: Developer ID does not exist!"
19      ELSE
20          PRINT "-------------------------------------------------------------------------------------------------"
21          PRINT "| ID          | Name                    | Languages                                | Salary
($)   |"
22          PRINT "-------------------------------------------------------------------------------------------------"
23
24          Join developer.programmingLanguages into a single string separated by comma
25          PRINT formatted row with (devID, fullName, languages string, salaryUSD)
26
27          PRINT "-------------------------------------------------------------------------------------------------"
28      END IF
29
30  END
```

Listing 4.3: Pseudocode for Search for a Developer by ID

4.4 Function 4: Update a Developer's salary by ID

```
1   ALGORITHM Update Developer Salary By ID BEGIN
2
3       PRINT "--- Update Developer Salary ---"
4
5       REPEAT
6           INPUT devID (format: DEVXXX)
7
8           IF devID format is invalid THEN
9               PRINT "Error: Invalid ID format! Format must be DEVXXX."
10          ELSE
11              BREAK LOOP
12          END IF
13      END REPEAT
14
15      Search developer record by devID in developerList
16
17      IF developer is NOT found THEN
18          PRINT "Error: Developer ID does not exist!"
19      ELSE
20          REPEAT
21              INPUT newSalary (Must be numbers)
22
23              IF newSalary format is invalid OR newSalary <= 1000 THEN
24                  PRINT "Error: Salary must be greater than 1000!"
25              ELSE
26                  BREAK LOOP
27              END IF
28          END REPEAT
29
30          SET developer.salaryUSD = newSalary
31          PRINT "[Success] Salary updated successfully."
32      END IF
33
34  END
```

Listing 4.4: Pseudocode for Update a Developer's salary by ID

4.5 Function 5: List all Developers by Language

```
1   ALGORITHM List All Developers By Language BEGIN
2
3       PRINT "--- List All Developers By Language ---"
4
5       INPUT searchLanguage (format: any non-empty string)
6
7       Initialize results list (empty)
8
9       FOR EACH developer IN developerList
10          Join developer.programmingLanguages into a single string separates by comma
11          IF developer's languages string contains searchLanguage (ignore case) THEN
12              Add developer TO results list
13          END IF
14      END FOR
15
16      IF results list is EMPTY THEN
17          PRINT "No developers found with language: " + searchLanguage
18      ELSE
19          PRINT "-------------------------------------------------------------------------------------------------"
20          PRINT "| ID          | Name                    | Languages                                | Salary
($)   |"
21          PRINT "-------------------------------------------------------------------------------------------------"
22
23          FOR EACH developer IN results list
24              Join developer.programmingLanguages into a single string separated by comma
25              PRINT formatted row with (devID, fullName, languages string, salaryUSD)
26          END FOR
27
28          PRINT "-------------------------------------------------------------------------------------------------"
29      END IF
30
31  END
```

Listing 4.5: Pseudocode for List all Developers by Language

4.6 Function 6: Add a new Project

```
1   ALGORITHM Add New Project BEGIN
2
3       PRINT "--- Add New Project ---"
4
5       REPEAT
6           INPUT projectID (format: PROJXX)
7           Search project in projectList by projectID
8
9           IF projectID format is invalid THEN
10              PRINT "Error: Invalid ID format! Format must be PROJXX."
11          ELSE IF project is found THEN
12              PRINT "Error: Project ID already exists!"
13          ELSE
14              BREAK LOOP
15          END IF
16      END REPEAT
17
18      REPEAT
19          Call listAllDevelopers() to display all developers
20          INPUT devID (format: DEVXXX)
21          Search developer in developerList by devID
22
23          IF developer is NOT found THEN
24              PRINT "Error: Dev ID does not exist!"
25          ELSE
26              BREAK LOOP
27          END IF
28      END REPEAT
29
30      INPUT projectName (format: any non-empty string)
31
32      INPUT projectDuration (Must be an integer >= 1)
33
34      REPEAT
35          INPUT startDay (format: dd/MM/yyyy)
36
```

```
37              IF startDay is NOT in the future THEN
38                  PRINT "Start day must be in the future!"
39              ELSE
40                  BREAK LOOP
41              END IF
42          END REPEAT
43
44          Create new Project object with (projectID, developer, projectName, projectDuration, startDay)
45          Add the new Project into projectList
46
47          PRINT "[Success] Add project successfully!"
48
49      END
```

Listing 4.6: Pseudocode for add new project

4.7 Function 7: List all Projects by Developer (Grouped)

```
1   ALGORITHM List All Projects By Developer BEGIN
2
3       PRINT "--- List All Projects By Developer ---"
4
5       IF developerList is EMPTY THEN
6           PRINT "No developers found."
7           RETURN
8       END IF
9
10      PRINT "-----------------------------------------------------------------------------------------------------"
11      PRINT "| ID          | Name                      | Languages                                  | Salary ($)
|"
12      PRINT "-----------------------------------------------------------------------------------------------------"
13
14      FOR EACH developer IN developerList
15          Join developer.programmingLanguages into a single string separated by comma
16          PRINT formatted row with (developer.devID, developer.fullName, languages string,
developer.salaryUSD)
17
18          Initialize flag = FALSE
19
20          FOR EACH project IN projectList
21              IF project.devID equals developer.devID THEN
22                  PRINT "Project: " + project.projectID + " " + project.durationMonths + " " + project.startDate
23                  SET flag = TRUE
24              END IF
25          END FOR
26
27          IF flag is FALSE THEN
28              PRINT "Not found project!"
29          END IF
30      END FOR
31
32      PRINT "-----------------------------------------------------------------------------------------------------"
33
34  END
```

Listing 4.7: Pseudocode for List all Projects by Developer (Grouped)

4.8 Function 8: Calculate Total Experience by Dev ID

```
ALGORITHM Calculate Total Experience By Dev ID BEGIN

    PRINT "--- Calculate Total Experience By Dev ID ---"

    REPEAT
        Call listAllDevelopers() to display all developers
        INPUT devID (format: DEVXXX)
        Search developer in developerList by devID

        IF developer is NOT found THEN
            PRINT "Error: Dev ID does not exist!"
        ELSE
            BREAK LOOP
        END IF
    END REPEAT

    Initialize sum = 0
    Initialize flag = FALSE

    Get currentDate = Current Date

    FOR EACH project IN projectList
        IF project.devID equals developer.devID THEN
            Calculate endDate = project.startDate + project.durationMonths

            IF endDate is before currentDate THEN
                sum = sum + project.durationMonths
            ELSE
                sum = sum + (current month - project.startDate's month)
            END IF

            flag = TRUE
        END IF
    END FOR

    IF flag is FALSE THEN
        PRINT "Developer does not join any project!"
    ELSE
        PRINT "Total experience: " + sum + " months."
    END IF

END
```

Listing 4.8: Pseudocode for Calculate Total Experience by DevID

4.9 Function 9: Remove a Developer by ID

```
1   ALGORITHM Remove Developer By ID BEGIN
2
3       PRINT "--- Remove Developer By ID ---"
4
5       Call listAllDevelopers() to display all developers
6       INPUT devID (format: DEVXXX)
7
8       Search developer record by devID in developerList
9
10      IF developer is NOT found THEN
11          PRINT "Error: Dev ID does not exist!"
12          RETURN
13      END IF
14
15      Initialize isAssigned = FALSE
16
17      FOR EACH project IN projectList
18          IF project.devID equals developer.devID THEN
19              SET isAssigned = TRUE
20              BREAK LOOP
21          END IF
22      END FOR
23
24      IF isAssigned is TRUE THEN
25          PRINT "Error: Cannot delete: Developer is assigned to projects."
26      ELSE
27          Remove developer from developerList
28          PRINT "[Success] Remove successful!"
29      END IF
30
31  END
```

Listing 4.9: Remove a Developer by ID

4.10 Function 10: Sort Developers by Salary

```
1   ALGORITHM Sort Developers By Salary BEGIN
2
3       PRINT "--- Sort Developers By Salary ---"
4
5       IF developerList is EMPTY THEN
6           PRINT "Empty List !"
7           RETURN
8       END IF
9
10      Sort developerList IN ASCENDING ORDER by developer.salaryUSD
11
12      // Bắt đầu gọi logic của hàm listAllDevelopers()
13      PRINT "-----------------------------------------------------------------------------------------------"
14      PRINT "| ID          | Name                    | Languages                                  | Salary ($)    |"
15      PRINT "-----------------------------------------------------------------------------------------------"
16
17      FOR EACH developer IN developerList
18          Join developer.programmingLanguages into a single string separated by comma
19          PRINT formatted row with (developer.devID, developer.fullName, languages string, developer.salaryUSD)
20      END FOR
21
22      PRINT "-----------------------------------------------------------------------------------------------"
23
24  END
```

Listing 4.10: Sort Developers by Salary

4.11 Function 11: Save data to files

```
1   ALGORITHM Save Data To Files BEGIN
2
3       PRINT "--- Save Data To Files ---"
4
5       // Đối với DeveloperList
6       INPUT developerFileName (format: string)
7       Call FileUtils.writeToFile(developerFileName, developerList)
8
9       // Đối với ProjectList
10      INPUT projectFileName (format: string)
11      Call FileUtils.writeToFile(projectFileName, projectList)
12
13      PRINT "[Success] Data saved to files successfully!"
14
15  END
```

Listing 4.11: Save data to files

4.12 Function 12: Quit program

```
1   ALGORITHM Save Configuration To File BEGIN
2
3       INPUT fileName (format: string)
4
5       Open file with fileName for writing
6
7       FOR EACH item IN currentList
8           Format item data into CSV specific string format
9           Write string format to the file
10      END FOR
11
12      Close the file
13
14      PRINT "[Success] Save successful!"
15
16  END
```

Listing 4.12: Quit program

Chapter 6 Business Logic Architecture

6.1.1 Developer Management Logic

The system follows these rules for managing software developers:

- **Unique ID Validation:** Every developer must have a unique ID following the format `DEVXXX` (e.g., DEV001). The system validates the format using Regex (`Validator.DEV_ID_REGEX`) and checks for duplicates by calling `searchById()`. If the ID already exists in `DeveloperList`, the system displays an error and prompts the user to re-enter.
- **Data Format Validation:**
- The developer's Full Name must contain only alphabetic characters and spaces, validated by `Validator.FULL_NAME_REGEX`.
- The salary must be a valid number and strictly greater than 1,000 USD (checked via `Validator.SALARY_REGEX`).
- **Flexible Programming Languages Input:** The system allows users to input multiple programming languages for a developer as a single comma-separated string. The logic automatically trims and splits this string into a standard List array before saving it to the developer object.
- **Search by Partial Match:** When searching for developers by a specific programming language, the system uses a case-insensitive search (`contains()`). It ignores letter cases, meaning searching for "java" or "JAVA" will return all developers who have Java in their language list.
- **Safe Developer Deletion:** A developer can only be removed if they are not currently assigned to any projects. Before deletion, the system calls `isDeveloperJoinProject()` to check associations in `ProjectList`. If the developer is actively participating in one or more projects, the deletion is immediately rejected to prevent orphaned project records.

6.1.2 Project Management Logic

The project allocation system uses the following rules to handle assignments:

- **Unique Project ID Validation:** Every project must have a unique ID following the format `PROJXX` (e.g., PROJ01). This is validated by `Validator.PROJ_ID_REGEX`. Duplicates are prevented by checking the existing records in `ProjectList`.
- **Linked to a Valid Developer:** A project must be assigned to an existing developer. When creating or loading a project, the system verifies the entered Developer ID against the `DeveloperList` using `developerList.searchById()`. If the developer does not exist, the project cannot be created.
- **Future Start Date Constraint:** The system strictly enforces that the Start Date of any new project must be in the future (i.e., strictly after the current system date, verified using `LocalDate.now()`). Projects cannot be scheduled starting in the past or on the very same day of creation.
- **Strict Duration Rules:** The duration of a project must be a positive integer strictly greater than 0 (minimum 1 month). The system uses `Inputter.getInt()` with parameterized boundaries to enforce this rule during user input.

6.1.3 Reporting and Analytics

The system provides reports and calculations to help managers evaluate developer contributions:

- **Sorting by Salary:** The system allows sorting all developers in ascending order based on their Salary (USD). This uses `Double.compare()` to organize the `DeveloperList` efficiently, helping HR and management quickly identify low to high-cost resources.
- **Project Grouping by Developer:** The system provides a comprehensive cross-reference report that lists all developers alongside the specific projects they are assigned to. It iterates through the `DeveloperList`, prints the developer's details, and then queries the `ProjectList` to list all associated project IDs, durations, and start dates right beneath the developer's name.
- **Dynamic Experience Calculation:** The system calculates the total real-world experience of a developer dynamically based on project statuses:
    - If a project has already ended (Start Date + Duration Months < Current Date), the system adds the full `Duration Months` to the total experience.
    - If a project is currently ongoing (Start Date is in the past, but the End Date has not been reached), the system calculates the elapsed time (Current Month - Start Month) and adds it to the total experience.
    - Projects scheduled entirely in the future do not contribute to the current experience points. This logic ensures the generated "Total Experience" metric is accurate up to the present day.

Chapter 7 Conclusion

The Software Developer Management application was successfully developed using Java. The system implements ten core functions covering three main areas: Developer Management (add, list all, list by programming language, search by ID, update salary, sort by salary, remove developer), Project Management (add project, list grouped by developer), and Analytics (calculate total real-world experience for a developer). Data persistence is achieved through text file I/O operations (using `FileUtils`), ensuring that all developer profiles, programming language associations, and project assignments are saved and loaded reliably across sessions to prevent any data loss.